Shared Core semantic reads over retained IR: base linearization, effective fields and instance slots

HyMeKo Core – PlanOnly discovery bundle

Created-at: 2026-09-17 15:49 +02:00

Status and authority

- **PlanOnly.** No protected Core source, dependency, fixture, lockfile or `CORE.YAML` item was edited. Implementation of step S2 requires the literal token `APPROVED-CORE-EDIT: ir-semantic-reads`, which has *not* been given. The earlier `ir-value-updates`, `ir-snapshot-install` and `ir-stream-cbor-snapshot` approvals do not cover this work.
- **ETA (after approval and decisions D1–D5).** Core S2: 3–4 h (one module of roughly 400 LOC plus about 30 tests, basis: `value_updates.rs`, 455 LOC including tests). Query/Geometry S3: about 2 h. Robotics S5: 2–3 h. Formula S4: about 2 h once unblocked. Simulator/XR repin S6: about 1 h (the published Simulator catalog build took 63 s).
- **Scope.** Discovery of which HyMeKo authoring semantics work, which depend on the consumer, and which are missing. The plan proposes an additive, borrowed-IR Core API for base relations, effective fields and instance-slot identity, plus migration order and acceptance cases. Out of scope: simulator/XR features, benchmarks, deployment, push to main, parser changes, a second IR, MessagePack, and text regeneration/reparse for runtime state.

1 Revision baseline

Fetched 2026-09-17 15:36 +02:00 (`git fetch -all`); Core refetched 15:48, still 0/0. Every checkout is 0/0 against its upstream. The Robotics and Kawamirai HTTP remotes failed authentication; their SSH `ls-remote` matches local HEAD. All of these are integration feature branches, not `main`. This is not an ecosystem merge or release, and a build of the full stack from a clean clone has not been demonstrated.

Checkout	Branch	HEAD	Notes
core	feat/ir-runtime-updates	2c4e072	parser pin f1162859; clean except untracked TeX byproducts
parser	master	be52aac	grammar surface identical to f1162859 (builder refactor only)
transformation	feat/robotics-authoring-integration	a0b5420	clean
transformation-formula	feat/formula-nurbs	7ef03ac	**other worker, uncommitted** (actuation, kinematic, contact, engine, visual)
graph	chore/published-ir-runtime	be8da9c	
akoire	feat/belief-planner	2e64c4b	
hive	feat/lan-model-catalog	e708ff6	
axon	feat/lan-ir-transport	ee6ae5d	
robotics	feat/robotics-belief-planner	9ec076c	untracked `texput.log`
kawamirai / hymeko_web / xr	feat/xr-passthrough-rendering / feat/scenario-runtime / feat/native-quest-workbench	5d022ba / be3da44 / d674d72	
simulator (xr-publish)	feat/quest-immersive-ui	4da2729	`[patch]` points at the dirty Formula worktree

Other open items: `core-parser-be52aac` and `core_cpp` carry other-worker changes. SomaChordex is not under Git. `computer-vision-modules` has no remote. `ecosystem.hymeko` lists 17 repositories and does *not* list `robotics` or `hystl`, even though the Robotics manifest consumes both. This is recorded as map drift and is not fixed here.

2 Evidence: semantics that already work

Basis key: R = read in code; P = Core CLI probe run in this task (Core release binary rebuilt at 2c4e072); T = existing test.

#	Capability	Location (file : symbol) and fact	Basis
E1	Base references on nodes and edges	`parser/src/hymeko.lalrpop`: `BaseRef`, `NodeBlockStart`, `EdgeBlockStart`. A base is signed `+ - ~` or unsigned (lowered as Neutral), and carries tags and an optional value.	R
E2	No template or instantiation syntax	The Description grammar has none; `? * ..` exist only for `QueryVar` and unused rules.	R
E3	Bases retained, never expanded	`ir/lower.rs`: `lower_node`/`lower_edge` → `NodeRec/EdgeRec.bases: Vec<SignedRefR>` in authored order with sign/tags/weights. An edge instance does not inherit base arcs (probe p8: 1 arc).	R,P
E4	Identity-preserving references	`resolution/resolve.rs`: `resolve_ref_to_declid` → `ValueR::Ref(DeclId)`. Lookup is lexical containment plus module-local `using`/import aliases. A match at more than one scope level is `AmbiguousRef` (p5), not shadowing. Base members are not reachable by path (p2b `UnresolvedRef inst.friction`). Duplicate paths are an error.	R,P
E5	Diamond imports share identity	`module_store.rs`: `linked_imports` links each canonical file once.	R
E6	Compile-time arithmetic	`resolution/const_resolve.rs`: `evaluate_consts`/`resolve_consts` handle forward refs and detect cycles, division by zero and undefined names; `pi`, `exp`. The IR stores only the folded `Num`.	R,P,T
E7	Signed incidences with roles	`ArcRec.refs` holds target, sign, tags, weights and `direction=Unspecified`. `ir/incidence.rs`: `IncidenceIndex`, `IncidenceRef` give reverse navigation; Graph `direct_ir` uses them (Robotics A*).	R,T
E8	Structural read API	`ir/access.rs` (total accessors); `canonical_hash::build_path_index` → `DerivedPathIndex` (path ↔ DeclId).	R
E9	Runtime value updates without parser	`module_store/execution/value_updates.rs`: `CompiledModel::with_value_updates` takes `Set/Text/Clear` on leaf Node declarations only. It is copy-on-write, keeps IDs/hierarchy/bases/index, and validates coherence and the model contract. Derived expressions are not re-evaluated. `ExecutionState::prepare_model_at/commit` reject stale or foreign bases.	R,T
E10	Canonical CBOR plus `ir_stream`	`writers/cbor_writer/stream.rs`: `encode_snapshot_messages`/`decode_snapshot`; assembler Begin→Chunk* →Commit; `CompiledModel::from_decoded_cbor`. HIVE `lan.rs` and `lib.rs::apply_value` use exactly these.	R,T

E11	Navigation example	`robotics/examples/cyberdisc/navigation.hymeko`: import/`using`, `+ <isa>` classification, a 3-point hyperedge region, shared profile references, per-instance `state`, const arithmetic. `navigation.rs::NavigationView::read` feeds Graph `astar_berge_path` and the AKOIRE `PlanGate`. `tests/navigation.rs`: detour 4 m→8 m through `ValueUpdate::Text` with no source read; a shared profile update affects every referencing region; reference rebinding; the `raduis_m` mutation is rejected as a missing field.	R,T
E12	ArUco observation state	`robotics/src/perception/recording.rs`: `RecordingSlots` requires authored leaf slots; each capture is applied as `ValueUpdate`s through `ExecutionState`. `perception.rs::source` emits text only as an export projection.	R,T

3 Consumer-dependent semantics (present, not shared)

#	Capability	Implementations and their divergence
C1	"X is-a Y"	Seven walks. Query `QueryEngine::check_inheritance`: by name, depth 8, not reflexive, tags ignored. `predicate_expr::decl_inherits`: visited set, reflexive. `rewrite/template.rs::decl_inherits_from`: depth 16. Geometry `check_inherits_simple`: depth 4; `geometry_decl`: direct bases only. WebXR `isa_transitive`: by DeclId, depth 1024, no deduplication. Dynamics/Optimize `Reader::inherits`: by name, visited set. None checks `<isa>` or the sign; all name-based variants are namespace-blind.
C2	Field read	Direct children only in Query `MatchContext::resolve_field_decl/get_field_at`, Geometry `find_child_*` and `Fields` (a0b5420), Robotics `pipeline::Schema`, Dynamics/Optimize `Reader::field`, WebXR `attr_value`, Graph octree `field`. `MatchContext` also turns `ValueR::Ref` into a name string, adds an edge-base fallback and a single-child rule.
C3	Inherited reference field	Geometry `find_inherited_ref_target` (committed): own children first, then DFS over bases, depth 4. A non-reference local `limit` is skipped. Used by joint limits only.
C4	General inherited field (Formula)	`transformation-formula .../actuation.rs::Fields::child`, **uncommitted**: stack DFS in authored base order, visited set, first name match wins, sign/tags ignored, no diagnostics. The Simulator's committed manifest patches to it. Diamond case: `x: A, B; A: R; B: R { f }`; the DFS returns R's `f` before B's override.

4 Gaps

G1 No Core base-relation API: no selection policy, no transitive is-a by DeclId, no cycle diagnosis. *Probe p1:* `a: + <isa> b; b: + <isa> a` *compiles.*

G2 No Core effective-field resolution, and at least six consumer copies with contradictory order, depth and reflexivity (C1–C4).

G3 No declared precedence among multiple bases and no ambiguity or diagnostic surface.

G4 No owner identity for an effective value. A writer cannot tell a shared profile slot from instance state. For example, `sphere_impact.a` and `.b` both resolve `link_geometry` to the `c.sphere_body` declaration; updating that DeclId changes both.

G5 No structural update. `with_value_updates` cannot allocate an instance slot or change bases, arcs or incidence weights (arc declarations are `HyperArc` and are rejected as `InvalidLeaf`). Mutable instance state must be authored as local slots.

G6 No field-path traversal that follows references while preserving identity.

G7 No field-schema validation. A misspelled override is simply a new local declaration (p2a `mas 4.0` compiles). This is not proposed as an automatic check.

G8 Base selection is ambiguous. The language guide says `<isa>` marks inheritance, but corpora use untagged mixins (`a: link, m.impact_blue, c.sphere_body`) and namespace-like bases (`impact: elements, geometry, axes`).

G9 Const hazards: a duplicate const silently keeps the last value (p4); a single-segment reference that matches a const name silently becomes a number (p3, `link -> ghost`); consts are module-local (p6); expressions keep no provenance, so there is no reactive recomputation.

G10 Templates are text rendering in Query `rewrite`. `regen::emit_split_rewrite` and `hymeko_emitter::emit_bases` drop base sign and tags (lossy projections).

G11 Ecosystem-map drift: `robotics` and `hystl` are absent from `ecosystem.hymeko`.

G12 Publication coupling: the committed Simulator 4da2729 depends on uncommitted other-worker code in the Formula worktree.

5 Proposed shared Core API

Owner: Core (capability `semantic_authority` and boundary `only_the_description_is_authoritative` in `ecosystem.hymeko`). New module `hymeko_core/src/ir/semantics.rs` (with private submodules `bases`, `fields`); `ir/mod.rs` declares it. The API is additive, read-only and derived. It borrows one `Ir`, is never serialized, and adds no dependency, CBOR, digest or hash change.

```
pub enum BaseSelection { All, Tagged(SymId) }
pub struct BasePolicy { pub selection: BaseSelection, pub max_depth: u16, pub max_entries: u32 }

pub struct BaseView<'ir> { /* &'ir Ir, policy, CSR linearizations */ }
impl<'ir> BaseView<'ir> {
    pub fn build(ir: &'ir Ir, policy: BasePolicy) -> Result<Self, BaseError>;
    pub fn direct_bases(&self, d: DeclId) -> impl Iterator<Item = &'ir SignedRefR> + '_;
    pub fn linearization(&self, d: DeclId) -> &[DeclId];          // starts with d
    pub fn is_a(&self, d: DeclId, class: DeclId) -> bool;          // reflexive, by DeclId
    pub fn field(&self, owner: DeclId, name: SymId) -> Option<FieldBinding<'ir>>;
    pub fn shadowed(&self, owner: DeclId, name: SymId) -> impl Iterator<Item = FieldBinding<'ir>> + '_;
    pub fn effective_fields(&self, owner: DeclId) -> impl Iterator<Item = FieldBinding<'ir>> + '_;
    pub fn path(&self, owner: DeclId, path: &[SymId]) -> Result<FieldBinding<'ir>, FieldError>;
    pub fn instance_slot(&self, owner: DeclId, name: SymId) -> Result<DeclId, SlotError>;
}
pub struct FieldBinding<'ir> {
    pub requested: DeclId,   // declaration that was asked
    pub declaration: DeclId, // slot supplying the value: the identity a ValueUpdate targets
    pub provider: DeclId,    // requested itself or a base in its linearization
    pub rank: u32,           // position of provider in the linearization (0 = local)
    pub value: Option<&'ir ValueR>,
}
impl FieldBinding<'_> {  // kind-checked, never falls back to a base
    pub fn number(&self) -> Result<f64, FieldError>;
    pub fn reference(&self) -> Result<DeclId, FieldError>;
    pub fn list(&self) -> Result<&[ValueR], FieldError>;
    pub fn text<'i>(&self, it: &'i Interner) -> Result<&'i str, FieldError>;
}
pub enum BaseError  { Cycle { path: Vec<DeclId> }, InconsistentOrder { declaration: DeclId },
                      DanglingBase { declaration: DeclId, target: DeclId }, LimitExceeded }
pub enum FieldError { Missing { owner: DeclId, name: SymId },
                      WrongKind { declaration: DeclId, expected: ValueKind, found: Option<ValueKind> },
```

```
                        ReferenceCycle { at: DeclId }, PathLimit }
pub enum SlotError   { Missing, NotLeaf { declaration: DeclId },
                        Inherited { declaration: DeclId, provider: DeclId } }
```

Normative rules

R1 **Base selection** is an explicit policy that becomes part of every result. `All` takes every base reference and preserves sign and tags without filtering, which matches the behavior of all existing consumers. `Tagged(isa)` takes only bases carrying that tag. There is no hidden default (decision D1).

R2 **Multiple inheritance order** is the C3 linearization over selected bases in authored order: $L(d) = d + \text{merge}(L(b_1), \ldots, L(b_n), [$ It is deterministic, puts a diamond ancestor after all its descendants, and fails closed with `InconsistentOrder`. Corpus check: `sphere_impact.a` gives $[a, link, meta_element, impact_blue, sphere_body, neutral, material]$, so `color` still comes from `impact_blue`, identical to today's Formula result.

R3 **Cycles** under the selected policy are detected at build time (DFS colouring) and returned as `BaseError::Cycle` with the DeclId path, and the whole view is refused. Compilation stays unchanged. Depth and entry limits return typed errors, never panics.

R4 **Field lookup** is by `SymId` among the children of each provider in linearization order. The first provider that declares the name wins, based on presence: a local `mass;` or `mass {}` shadows with `value=None` and does not fall back. A local block replaces an inherited block as a whole (no deep merge). A wrong-kind local value yields `FieldError::WrongKind`, never the base value. `shadowed` exposes the hidden providers for diagnostics. Incomparable providers are ordered by C3, not rejected, because strict rejection would break `sphere_impact` (`impact_blue` vs `neutral`).

R5 **Nested inherited blocks** keep the provider's identity. `path(a,[link_geometry,dimension])` resolves inside the `c.sphere_body.link_geometry` declaration, using that block's own linearization. The instance gets no private copy.

R6 **References**: a stored `ValueR::Ref(DeclId)` is returned unchanged and is not retargeted to same-named instance declarations. `path` follows a reference only on intermediate segments; the terminal segment returns the binding that holds the reference. A reference cycle inside a path is a typed error.

R7 **Instance state**: `instance_slot` succeeds only for a local leaf Node. An inherited effective binding returns `SlotError::Inherited`. Producers of observation or belief state (Robotics perception, AKOIRE domain adapters, HIVE clients) obtain slots through it. A shared-profile edit names the profile declaration explicitly.

R8 **Validity after changes**: linearizations and binding DeclIds depend only on hierarchy, names and bases, all of which `with_value_updates` preserves (E9). Values are read from the borrowed `Ir`. Version 1 rebuilds the view for each `ExecutionSnapshot`; no cache is added without an explicit measurement request. Rebinding a reference changes `path` results on the successor only. Recompilation or snapshot installation requires a rebuild, and the borrow lifetime prevents mixing IRs.

R9 **Non-goals**: no schema validation, default expansion, materialization, text generation or expression recomputation, and no change to compilation, CBOR, `ir_stream`, V1 hash or semantic digest.

R10 **Portability**: `Vec`/slice data only, no filesystem, thread or clock use, no rayon, so it targets native and `wasm32`.

Rejected alternatives

- A shared helper inside Transformation/Query: Query is not the semantic owner, and Robotics, AKOIRE and HIVE must not depend on Transformation for meaning.
- Expanding inherited fields into the IR during lowering: it changes IR, CBOR and digests, destroys shared-profile identity, and contradicts the authoring contract's sharing principle.
- Formula DFS order: the diamond counterexample in C4.
- Strict ambiguity errors, or an `<isa>`-only default: both break the existing Simulator data.

6 Migration order (provider → consumer)

S0 Done in this task: discovery, this bundle and the local report. No source change.

S1 User decisions D1–D5 (below). Approval token for S2.

S2 **Core**: implement the module, unit tests in the module, integration test `hymeko_core/tests/semantic_reads.rs` (inline `MemProvider` sources; `data/**` is locked), `SYMBOLS.md`, approval record in `CORE.YAML`, correctness gates A1–A13. Branch from freshly fetched Core according to D3.

S3 **Transformation primary**: add `Predicate::isa(DeclId)` backed by `BaseView`; keep the name-based predicate with documented limits; add an effective-field mode to `MatchContext`; replace `find_inherited_ref_target` and `check_inherits_simple`. Pin the Core revision. Gates A14–A15.

S4 **Formula worktree**: only after the other worker publishes or hands over. Replace `Fields::child` with `BaseView::field`. Gate A16 (differential).

S5 **Robotics**: `pipeline::Schema` delegates to `BaseView`; navigation profile defaults and the camera-recording marker profile are refactored only with tests; recording acquires slots via `instance_slot`. Gates A17–A18.

S6 **Simulator/XR/Web**: repin the published Geometry/Query revision, remove the dirty path patch, rerun existing catalog/robot tests. AKOIRE, HIVE, Axon, Graph, Kawamirai: no API change.

7 Acceptance cases (correctness only; no benchmark)

A1	Diamond `x: A, B; A: R; B: R`; `f` authored on R and B ⇒ linearization $[x, A, B, R]$, provider B (discriminates C3 from the Formula DFS).
A2	`X: A, B; Y: B, A; Z: X, Y` ⇒ `InconsistentOrder`.
A3	Probe p1 base cycle ⇒ `BaseError::Cycle [a,b]`; the same source still compiles.
A4	Unsigned, `-`, `~ <role>` and `+ <isa>` bases under `All` and `Tagged(isa)`: `is_a` as specified; sign and tags preserved in `direct_bases`.
A5	Local override value; valueless local shadows (`None`); local block replaces inherited block; wrong kind ⇒ `WrongKind`, and `shadowed` lists the base.
A6	Misspelled local `mas` ⇒ `field(mass)` returns the inherited provider, no error (documents the absence of schema validation).
A7	`path(navigation,[regions,central,clearance,radius_m])`.declaration = DeclId of `profiles.narrow.radius_m`; an inherited reference is not retargeted; a reference cycle gives a typed error.
A8	Instance isolation: two instances with a shared base `state` ⇒ `instance_slot` gives `Inherited`; with local slots the DeclIds are distinct; a `ValueUpdate` on inst1 leaves inst2 and the profile unchanged; a profile update changes every non-overriding inheritor; the nested shared block reports the mixin as provider.
A9	View on base vs `with_value_updates` successor: identical linearizations and binding DeclIds; reference rebinding changes `path` on the successor only; the old view is unchanged.
A10	`encode_compiled`→`decode`→`from_decoded_cbor` and `encode_snapshot_messages`→`decode_snapshot` (Begin/Chunk/Commit): identical linearizations, bindings and values.
A11	Counted parser: no parser call during build, reads or updates.
A12	Malformed IR (dangling base, oversize depth) ⇒ typed errors, no panic.
A13	`cargo check -p hymeko_core -target wasm32-unknown-unknown -locked` and the existing `hymeko_web` wasm document check.
A14	Query differential: `Predicate::inherits(name)` vs `BaseView::is_a` on Transformation fixtures with unique names; a name-collision fixture documents the difference.
A15	Geometry `per_joint_limits_resolved_through_the_limit_ref` unchanged with the Core resolver.
A16	Formula `Fields::child` vs `BaseView::field` on `meta_contact`, `meta_materials`, `sphere_impact`, `contact_cycle`, `contact_probe`: identical effective values; Simulator `ll_sphere_impact` and `tests/catalog.rs` unchanged.
A17	Robotics navigation with a profile default plus override: routes 4 m and 8 m unchanged, provider read counts unchanged, `regions.*.state` gives `instance_slot` Ok, `raduis_m` still rejected.
A18	Robotics recording: a marker whose `state` is inherited from a profile fails closed via `instance_slot`.

8 Interface changes, protected items, budget, rollback

- Additive public API only; no existing signature, wire format, hash or digest changes; no dependency change.
- `CORE.YAML` items touched in S2: `hymeko_core/src/ir/mod.rs`, new `hymeko_core/src/ir/semantics*.rs` (lockdown `full`) ⇒ approval required. `hymeko_core/tests/**` is allowlisted.

- Performance: not measured (no authorization). Design bound: build $O(D + \sum |L(d)|)$ under explicit limits; a field read is $O(|L(d)| \cdot \text{children})$. It is rebuilt per snapshot at Robotics input limits (4096 declarations).
- Rollback: additive module; consumers stay pinned to earlier revisions; revert per repository in consumer → provider order.

9 Risks

- DFS→C3 changes diamond results (A1, A16 detect it); namespace-like untagged bases create pseudo-fields under `All`.
- Name-based Query predicates stay namespace-blind until migrated; the WebXR walk has no deduplication.
- Formula ownership: editing or committing another worker's uncommitted `Fields::child` is out of scope.
- Writers that update a shared provider by mistake: mitigated only when they adopt `instance_slot` (A8, A18).
- Const diagnostics (G9) would change compile behavior and need a separate decision.

10 Open decisions for the user

D1 Base selection per consumer: `All` (compatible with the current corpus) vs `Tagged(isa)` (as written in the language guide), and whether untagged namespace bases should be re-authored.
D2 Multiple-inheritance order: C3 (recommended) vs the current DFS.
D3 Core approval token `APPROVED-CORE-EDIT: ir-semantic-reads`, and the target branch/base for S2.
D4 Coordination of the Formula worktree owner, the `core-parser-be52aac` and `core_cpp` work, the SomaChordex repository address and the `computer-vision-modules` remote.
D5 Whether G9 const diagnostics and G11 map drift become separate owner tasks.

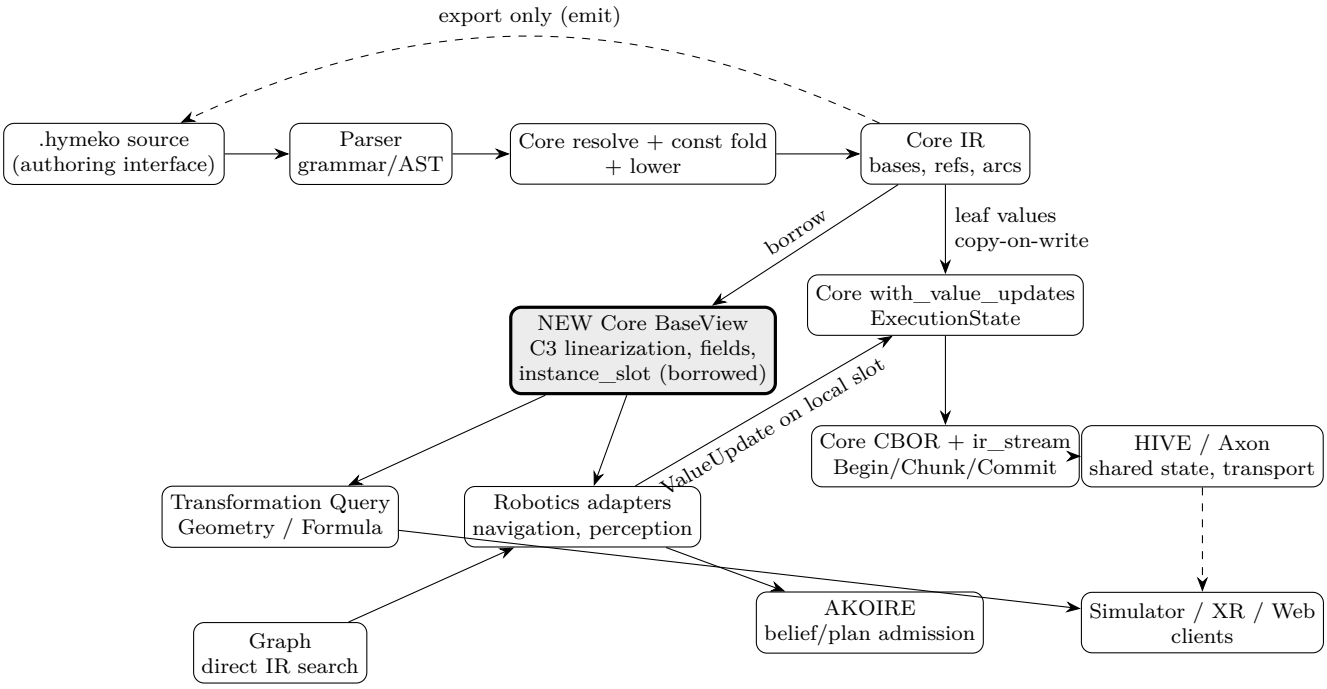


Figure 1: Ownership and data flow; the only new element is the Core `BaseView`.